

Shopify Store Analytics Dashboard

Enterprise E-Commerce Performance Intelligence & Catalog Analytics Manual

A full-stack, production-grade analytics platform and merchant control dashboard built with the modern MERN stack (React 19, Node.js, Express, MongoDB Atlas). Features high-performance MongoDB aggregation pipelines, real-time date range filters, multi-type interactive data visualizations, catalog velocity tracking, and Indian Rupee (INR) currency standardization.

Project Name	Shopify Store Analytics Studio
Store Name / Owner	Apex Retailers (Devicharan Prajapati)
Core Architecture	MERN Stack (React 19, Express.js, Node.js, MongoDB Atlas)
State Management	Redux Toolkit (@reduxjs/toolkit, react-redux)
Visualization Suite	Recharts (Area, Dual Bar, Horizontal Bar, Line, Donut)
Styling & Theme	Tailwind CSS v4 (Clean White & Emerald #10B981)
Base Currency	Indian Rupee (INR / Rs.) with en-IN formatting
Date Filter Range	Presets (Today, 7d, 30d, 90d, 1y) & Custom Boundaries
GitHub Repository	https://github.com/DevicharanPrajapati/ShopifyAnalyticsDashboard.git
Build Release	v1.2.0 Production Release

Key Architecture Takeaway: Designed strictly as a single-merchant powerhouse for Apex Retailers, this project eliminates retail guesswork by marrying MongoDB Atlas multi-stage aggregation pipelines with client-side reactive charts. It handles multi-range financial trend modeling, inventory stock comparison, and customer basket-size distributions with sub-second response times.

1. Executive Summary & System Architecture

Modern e-commerce merchants require instant, multi-faceted visibility into transaction volume, shopper behavior, and inventory velocity. The **Shopify Store Analytics Dashboard** provides a responsive, robust control tower that converts raw transactional data into high-value financial intelligence.

Decoupled Multi-Tier Architecture

- **1. Client Layer (SPA):** Built on React 19 and Vite. Utilizes Redux Toolkit for centralized analytics and UI collapse state caching, Recharts for responsive SVG visualizations, and Tailwind CSS v4 for clean emerald styling.
- **2. API & Routing Layer:** Node.js and Express.js REST API with standardized request wrappers (asyncHandler), custom response contracts (ApiResponse), and centralized error handling (ApiError).
- **3. Service & Analytics Engine:** Business logic powered by native MongoDB multi-stage aggregation pipelines (\$match, \$unwind, \$group, \$bucket, \$lookup, \$project).
- **4. Database & Storage Layer:** MongoDB Atlas cloud cluster with compound indexes optimized for date-bounded queries across orders, products, and daily visitor footfall.

Complete Technology Stack

Domain	Technology	Role & Purpose in Project
Frontend Core	React 19 + Vite	High-speed reactive SPA with instant HMR and optimized bundle splitting
Global State	Redux Toolkit	Centralized date filtering, overview metrics, and UI collapse persistence
Visualizations	Recharts	Area charts, dual bar charts, horizontal bars, lines, and donut distributions
Styling Engine	Tailwind CSS v4	Utility-first layout, custom emerald palette, and fully responsive breakpoints
Backend Runtime	Node.js (v22) + Express	Lightweight, asynchronous REST API serving JSON endpoints
Database	MongoDB Atlas (Mongoose)	Document storage with aggregation pipelines, indexes, and schema validation
HTTP Client	Axios	Configured instance with base URLs, timeouts, and automated serialization
Iconography	Lucide React	Consistent visual icons for navigation, KPI cards, and header toggles

2. Application Modules: Dashboard & Orders

2.1 Store Performance Dashboard (/)

The Dashboard serves as the central operational hub, loading 7 analytics streams in parallel via `/api/analytics/dashboard`:

- **Shopify-Style Interactive KPI Cards:** Four responsive cards displaying Total Revenue, Total Orders, Conversion Rate, and Average Order Value (AOV). Clicking on ANY card highlights it as active and dynamically renders that metric in the primary chart below.
- **Dynamic Multi-Style Performance Chart:** Morphs based on the active KPI card. Features a dedicated Chart Type switcher allowing merchants to toggle between **Area Chart** (smooth gradient), **Bar Chart** (discrete columns), and **Line Chart** (trend stroke & points) with localStorage persistence.
- **Traffic vs Orders Trend (Dual Bar Chart):** Side-by-side comparison of daily store footfall (visitors) versus order conversions, revealing buyer purchase intent patterns across weekdays vs weekends.
- **Category Sales Distribution (Horizontal Bar Chart):** Ranks category revenue contribution (Electronics, Apparel, Home & Kitchen, Accessories, Bags, Footwear).
- **Payment Settlement Breakdown (Donut Chart):** Visualizes paid, pending, and refunded transactions with an interactive center counter.
- **Top Selling Products List:** Live ranking of top 5 revenue-generating items with SKU codes, unit sales, and gross revenue in INR.
- **Recent Orders Feed:** Live transaction table showing order ID, customer details, payment status badges, and transaction date.

2.2 Orders Analytics & Management (/orders)

Combines deep aggregate financial analysis with complete transaction fulfillment workflows:

- **Date Range Filter:** Merchants can toggle between presets (Today, Yesterday, 7d, 30d, 90d, 1y) or select custom date boundaries without page reloads.
- **Orders KPI Cards:** Orders in Range, Net Order Value (INR), Average Order Value (AOV), and Fulfilled Orders count with percentage fulfillment rate.
- **Order Basket Value Tiers (Bar Chart):** Uses MongoDB's \$bucket aggregation stage to classify orders into basket size tiers (Under Rs. 2,500; Rs. 2,500 - Rs. 5,000; Rs. 5,000 - Rs. 10,000; Rs. 10,000 - Rs. 50,000; Above Rs. 50,000).
- **Daily AOV Trend (Line Chart):** Tracks customer purchasing power fluctuations day-by-day.
- **Filter Tabs & Search:** Status tabs (All, Paid, Pending, Refunded) combined with text search across order ID, customer name, and email.
- **Dual View Layout:** Renders an expansive desktop table with payment badges, and automatically switches to touch-optimized order cards on mobile screens.

3. Products Module & Data Engineering

3.1 Products Catalog & Stock Velocity (/products)

- **Inventory KPI Cards:** Catalog SKUs count, Physical Units in Warehouse, Units Sold in period, and Top Performing Category with gross revenue contribution.
- **Category Sales Share (Donut Chart):** Visualizes category revenue share percentages with interactive hover cards.
- **Inventory Stock vs Units Sold (Dual Bar Chart):** Side-by-side comparison of shelf stock remaining vs units sold for bestsellers, alerting merchants to potential stockouts.
- **Product Catalog Grid:** Responsive 4-column card grid displaying product imagery, SKU codes, stock counts, retail prices in INR, and profit margin percentages calculated dynamically.

3.2 Database Models & Indexing Strategy

Mongoose schemas are engineered to support low-latency analytical aggregations over large transaction sets:

Model	Key Fields & Subdocuments	Index Architecture
Order (orders)	orderNumber, customer: { name, email, city }, items: [{ product, title, price, quantity }], totalAmount, subtotal, tax, financialStatus, fulfillmentStatus, orderDate	• Compound: { storeId: 1, orderDate: 1, financialStatus: 1 } • Single: orderNumber: 1
Product (products)	title, sku (unique), category, price (INR), costPrice (INR), inventoryQuantity, status (active/draft), image	• Text: { title: 'text', category: 'text' } • Single: category: 1, storeId: 1
VisitorTraffic (visitortraffics)	storeId, date, visitorsCount, sessionsCount	• Unique Compound: { storeId: 1, date: 1 }

3.3 Synthetic Data Seeding Engine

A realistic seeding script (server/src/seeds/seedData.js) populates 30 consecutive days of orders and traffic. It generates authentic Indian customer personas (Mumbai, Bengaluru, Delhi), realistic market pricing in INR (Rs. 1,499 - Rs. 12,999), and weekend-adjusted footfall delivering a realistic 2.5% - 4.5% conversion rate.

4. Backend REST API Reference

All endpoints adhere to REST conventions, returning standard HTTP codes and JSON payloads encapsulated by ApiResponse:

Method	Route Path	Query Parameters	Description
GET	/api/health	None	Service health status and UTC timestamp
GET	/api/analytics/overview	preset, startDate, endDate	KPI cards with % comparative growth
GET	/api/analytics/revenue-trend	preset, startDate, endDate	Daily revenue and order volume series
GET	/api/analytics/orders-stats	preset, startDate, endDate	Order value tiers and daily AOV trend
GET	/api/analytics/products-stats	preset, startDate, endDate	Category shares and stock vs units sold
GET	/api/analytics/dashboard	preset, startDate, endDate	Consolidated payload of all dashboard widgets
GET	/api/orders	status, search, page, limit	Paginated order list with search filters
GET	/api/orders/:id	None (path parameter)	Single order details by MongoDB ObjectId
GET	/api/products	category, search, page, limit	Paginated catalog products with search filter
GET	/api/products/:id	None (path parameter)	Single product details by MongoDB ObjectId

Standardized JSON Envelopes

Success Envelope (ApiResponse)	Error Envelope (ApiError)
<pre>{ "statusCode": 200, "success": true, "message": "Complete dashboard analytics retrieved", "data": { "overview": { "current": { ... } }, "revenueTrend": [...], "categorySales": [...] } }</pre>	<pre>{ "statusCode": 404, "success": false, "message": "Order not found: invalid ID format", "errors": [] }</pre>

5. Architectural Patterns & UI/UX Design System

5.1 Architectural Patterns & Utilities

- **asyncHandler Wrapper (server/src/utls/asyncHandler.js):** Eliminated repetitive try/catch blocks across every controller action. Automatically catches rejected promises and pipes them to Express's next() handler.
- **Centralized dateHelper (server/src/utls/dateHelper.js):** Provides startOfDay(), endOfDay(), calculatePercentageChange(), and getDateRangeFromQuery(). Standardizes time zone parsing and comparative period generation into a single call.
- **Defensive Input Validation:** Integrated mongoose.isValidObjectId() guards to eliminate unhandled CastErrors when querying by ID, gracefully returning HTTP 404 instead of HTTP 500 crashes.
- **Safe Pagination Clamping:** Clamps user pagination parameters with upper and lower bounds (Math.max(1, page) and Math.min(100, limit)), protecting the database from malicious unbounded queries.
- **Mongoose Lean Read Operations:** Employs .lean() on read-only queries to bypass full Mongoose document instantiation, drastically reducing memory overhead and JSON serialization latency.

5.2 UI/UX Design System & Navigation

- **Clean Shopify Emerald Theme:** Restored the crisp white and emerald green palette (#10B981) authentic to the Shopify merchant aesthetic, featuring clean slate backgrounds and borders.
- **Indian Rupee (INR) Standardization:** All currency representations across cards, charts, and tables display formatted in Indian Rupees (₹ / Rs.) with Indian number system comma grouping (e.g. Rs. 7,48,282).
- **Stationary Fixed Desktop Sidebar:** Desktop sidebar is anchored with fixed positioning (fixed inset-y-0 left-0 w-64 z-30), eliminating scrolling misalignment during long page navigation.
- **Sidebar Hide/Show Collapse Mode:** Users can toggle the sidebar closed or open via the header icon or the standard Ctrl+B (Cmd+B) keyboard shortcut. When collapsed, the main content area smoothly transitions to full-width (lg:pl-0). State persists in localStorage.
- **Custom Vector Logo Component:** Engineered a pure vector SVG logo component featuring an emerald shopping bag embedded with an upward-trending growth chart line and data points. Also exported as the browser favicon.
- **Minimalist Clean Footer:** Streamlined footer to a single minimal line with copyright and store identity, removing extraneous clutter.

6. Setup, Installation & Verification Guide

Prerequisites

• Node.js version 18.0.0 or higher • npm or yarn • MongoDB Atlas cluster or local MongoDB (v6.0+) • Git

Step-by-Step Installation

Step 1: Clone Repository

```
git clone https://github.com/DevicharanPrajapati/ShopifyAnalyticsDashboard.git
cd ShopifyAnalyticsDashboard
```

Step 2: Server Environment

Create server/.env with:

```
PORT=5000
MONGODB_URL=mongodb+srv://:@cluster0.mongodb.net/shopify_store_analytics
CLIENT_URL=http://localhost:5173
```

Step 3: Install & Seed

```
cd server && npm install
npm run seed # Populates 8 products, 138 orders in INR, 30 days traffic
```

Step 4: Launch Server

```
npm run dev # Runs Express API on http://localhost:5000 with nodemon
```

Step 5: Client Setup

```
cd ../client && npm install
npm run dev # Launches Vite development server on http://localhost:5173
```

Step 6: Production Build

```
npm run build # Compiles production-ready bundle into client/dist
```

Project Completion & Status

All features, endpoints, visualizations, responsive layouts, and server optimizations have been verified and pushed to the main branch of the GitHub repository. The project is completely functional and ready for evaluation.

Author / Developer: Devicharan Prajapati
Project: Shopify Store Analytics Dashboard
Store Identity: Apex Retailers (Single Store Architecture)
Repository: <https://github.com/DevicharanPrajapati/ShopifyAnalyticsDashboard.git>
Status: All Requirements Completed & Verified on Main Branch